COMPLETE VISUAL GUIDE

Build Your App on Replit

Manual coding · No AI · Step-by-step with screenshots & arrows

Ø=Üø Real screenshots

Ø=Ü» Full code examples

Ø=Ý Keyboard shortcuts

Ø=Þ€ Deploy live

CONTENTS

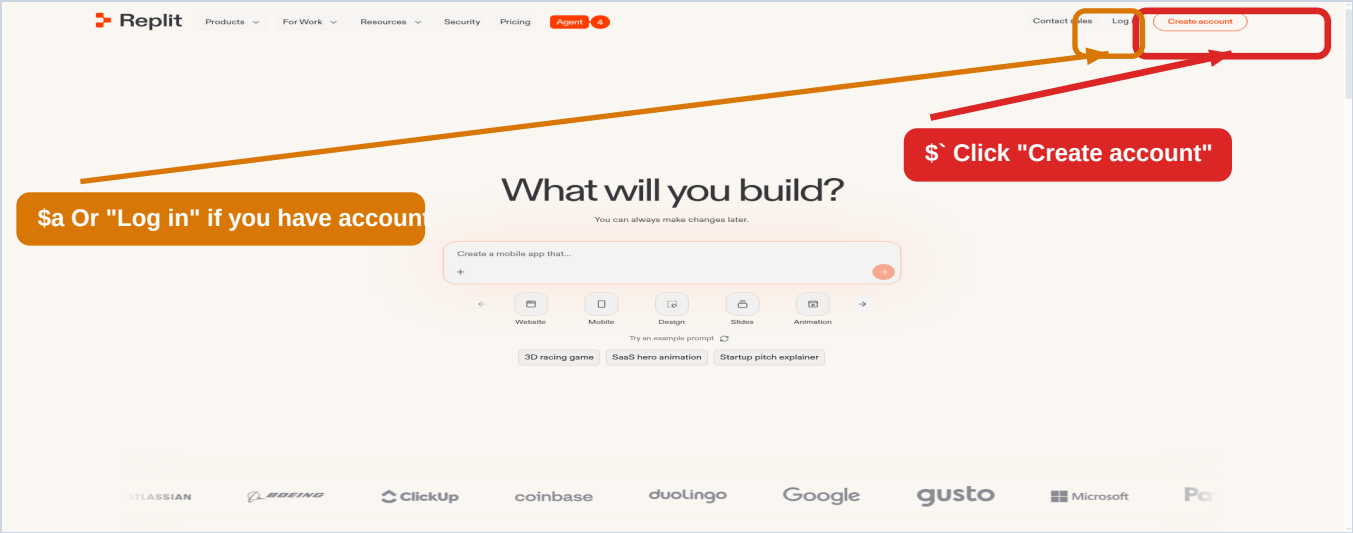
01 Create Your Replit Account

STEP 1 — Go to replit.com in your browser

Open any web browser (Chrome, Firefox, Safari, or Edge). Click the address bar at the top of the browser window and type:

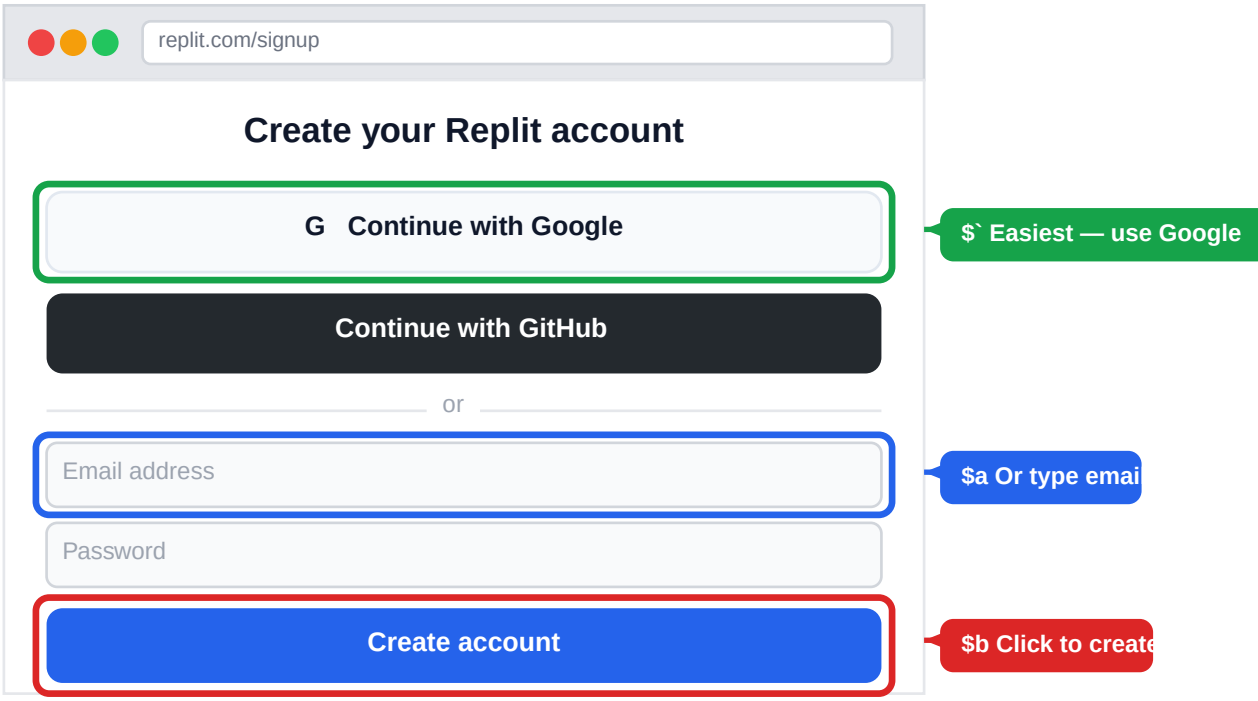
```
https://replit.com
```

Press Enter. You will see the Replit homepage. Below is a real screenshot — look at the TOP RIGHT corner:



Ø=Üi "Create account" is at the TOP RIGHT corner. Use Google login — it's the fastest, no new password needed.

STEP 2 — Sign up form: what to click



STEP 3 — Confirm email & choose free plan

1. **Check your email inbox**
   Replit sends a confirmation email. Open it (Gmail, Outlook, etc.).

2. **Click the blue link inside the email**
   This takes you back to Replit and activates your account.

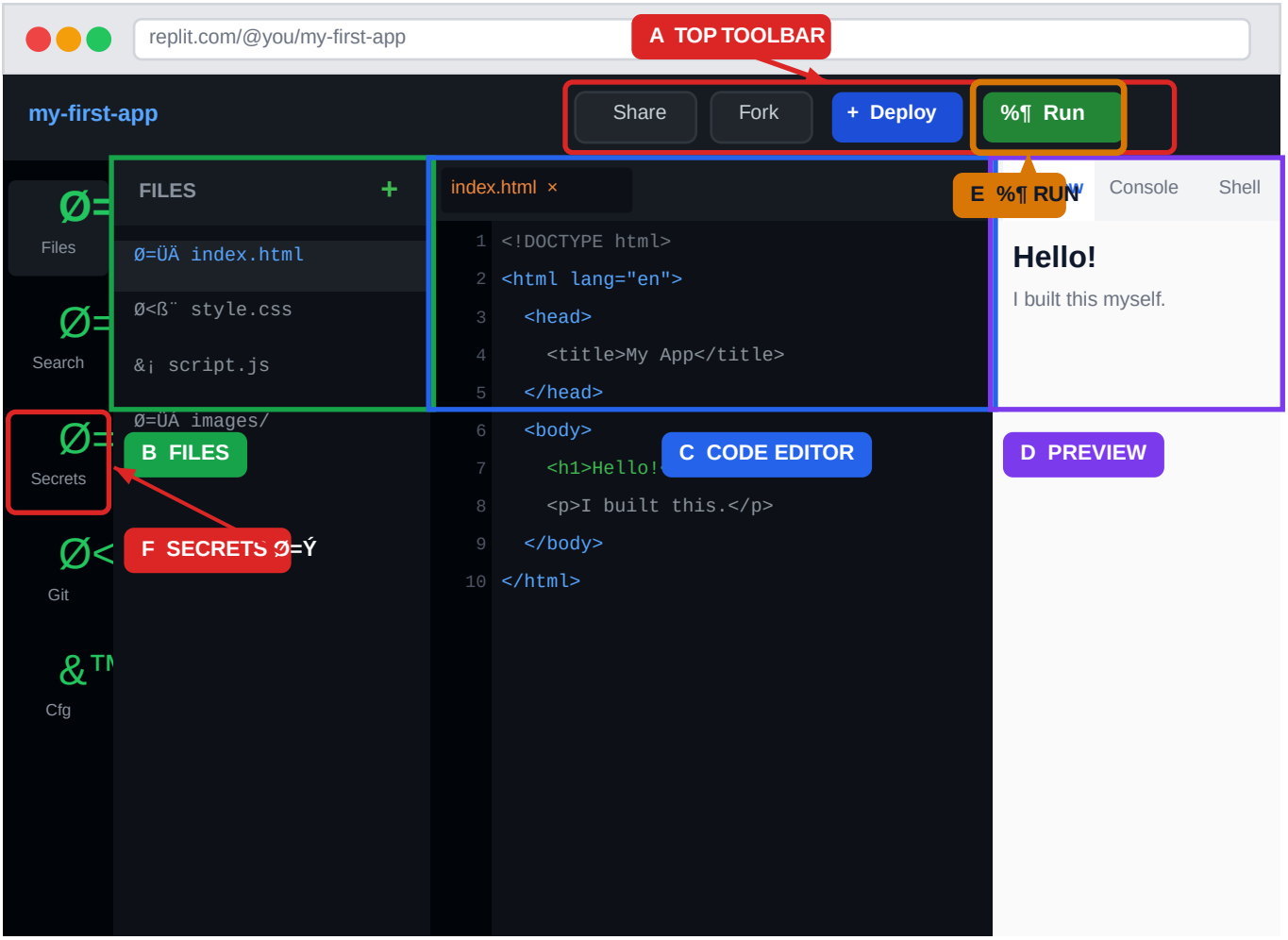
3. **Choose the free plan**
   Click "Start for free". You do NOT need to pay to start building real apps.

01

The Replit Editor

The Editor has 4 main areas — every one is labelled below

When you open any project, this is what you see. The coloured letters A–E match the legend below the diagram.



What each labelled area does

A Top Toolbar — RUN, DEPLOY, SHARE buttons. Click %¶ Run every time you want to test your code.

B File Explorer — all your project files. Click a file to open it. Click + to create a new file.

C Code Editor — this is where you TYPE your code. Line numbers are on the left.

D Preview tab — see your running app. Console tab — see errors. Shell tab — type commands.

E Run Button %¶ — starts your app. Shortcut: Ctrl + Enter (same result).

F Secrets Ø=Ý — store API keys and passwords here. NEVER put them in your code files.

02

Write Your First Code

CREATE a new project first

1 Click "+ Create Repl" on your dashboard

Go to the Replit homepage. Click the big blue "+ Create Repl" button on the left sidebar.

2 Search for "HTML, CSS, JS"

Type it in the search box. Select that template — it gives you a blank webpage, perfect for beginners.

3 Type a name like "my-first-app" and click Create

Replit builds your project in about 5 seconds and opens the editor.

OPEN index.html and type this HTML

In the File Explorer (left panel), click "index.html". Select all text (Ctrl + A) and delete it. Then type this — do not paste it, TYPING it builds the skill:

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>My First App</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>

  <h1>Hello, World!</h1>
  <p>I built this myself on Replit.</p>
  <button id="myBtn">Click me</button>

  <script src="script.js"></script>
</body>
</html>
```

Ø=Ü¾ Press Ctrl + S to SAVE, then click %¶ Run. In the Preview tab you should see "Hello, World!" — plain text for now, we will style it next.

CREATE style.css — add colours

In the File Explorer, click the + icon !’ "New File" !’ type "style.css" !’ press Enter. Then type:

```css
/* Controls how the page looks */
body {
  font-family: Arial, sans-serif;
  background-color: #0f172a;
  color: #f1f5f9;
  text-align: center;
  padding: 60px 20px;
}
h1 {
  font-size: 2.5rem;
  color: #3b82f6;
}
p  { color: #94a3b8; font-size: 1.1rem; }
#myBtn {
  background: #3b82f6;
  color: white;
  border: none;
  padding: 12px 28px;
  font-size: 1rem;
  border-radius: 8px;
  cursor: pointer;
  margin-top: 20px;
}
#myBtn:hover { background: #1d4ed8; }
```

CREATE script.js — add a click action

Click + icon !’ "New File" !’ type "script.js" !’ press Enter. Then type:

```javascript
// Get the button element
const btn = document.getElementById('myBtn');

// When the button is clicked, run this:
btn.addEventListener('click', function () {
  alert('You clicked the button! Ø<ß‰');
});
```

Save all files (Ctrl + S on each tab), click %¶ Run. Preview shows a dark page, blue heading, grey text, and a blue button. Click the button — an alert pops up.

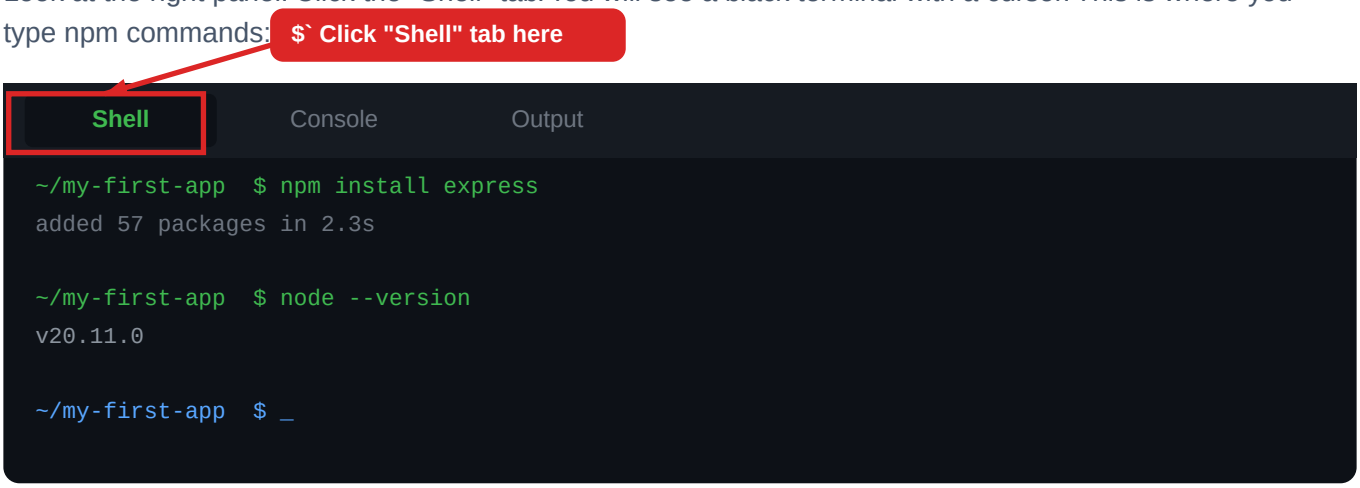
03 Installing Packages

What is a package?

A "package" (also called a library or module) is code someone else already wrote. Instead of building everything yourself, you install a package and use it. For example: the "express" package builds a web server in 5 lines. Without it, you would need 200 lines.

The Shell tab — where you type commands

Look at the right panel. Click the "Shell" tab. You will see a black terminal with a cursor. This is where you type npm commands:



npm — Node.js Package Manager commands

Type these in the Shell tab. Press Enter after each command to run it:

Install a package (example: express)

```
$ npm install express
```

Install multiple packages at once

```
$ npm install axios dotenv lodash
```

Install a dev-only package (only needed while coding)

```
$ npm install --save-dev nodemon
```

Remove a package you no longer need

```
$ npm uninstall express
```

See all installed packages

```
$ npm list --depth=0
```

Run the "start" script from package.json

```
$ npm run start
```

Run the development server (auto-reload on save)

```
$ npm run dev
```

> Ø=Ü¡ After running npm install, a folder called "node_modules" appears in your file explorer. Never edit anything inside it — it is auto-generated and can be re-created any time.

pip — Python Package Manager (for Python projects)

Install a package

```
$ pip install flask
```

Install from a requirements.txt file

```
$ pip install -r requirements.txt
```

See all installed packages

```
$ pip list
```

The package.json file

For Node.js projects, "package.json" is your project's main config file. It lists what packages your project needs. It is created automatically — never delete it.

package.json

```
{
  "name": "my-first-app",
  "version": "1.0.0",
  "scripts": {
    "start": "node server.js",
    "dev":   "nodemon server.js",
    "build": "vite build"
  },
  "dependencies": {
    "express": "^4.18.2"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

Build a Full Website

Your project will have these 6 files

```
my-website/
% % %  index.html     !• landing page
% % %  signup.html    !• create account
% % %  login.html     !• log in
% % %  style.css      !• shared styles
% % %  auth.css       !• form styles
% % %  auth.js        !• form logic
```

index.html — Landing Page (complete code)

HTML — index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>MyApp — Build Your Future</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>

  <!-- NAVIGATION BAR -->
  <nav class="navbar">
    <div class="nav-logo">MyApp</div>
    <ul class="nav-links">
      <li><a href="#">Home</a></li>
      <li><a href="#">Features</a></li>
    </ul>
    <div class="nav-btns">
      <a href="login.html"  class="btn-outline">Log in</a>
      <a href="signup.html" class="btn-primary">Sign up free</a>
    </div>
  </nav>

  <!-- HERO SECTION -->
  <section class="hero">
    <h1>Build something <span class="hl">amazing</span></h1>
    <p>The simplest way to create and deploy your app.</p>
    <a href="signup.html" class="btn-primary btn-lg">
      Get started — it's free
    </a>
  </section>

  <!-- FEATURES -->
  <section class="features">
    <div class="card"><div class="icon">Ø=Þ€</div><h3>Fast</h3><p>Deploy in seconds.</p></div>
    <div class="card"><div class="icon">Ø=Ý </div><h3>Secure</h3><p>Your data is safe.</p></div>
    <div class="card"><div class="icon">Ø=Ü¡</div><h3>Simple</h3><p>No experience needed.</p></
div>
  </section>

  <!-- FOOTER -->
  <footer><p>© 2025 MyApp. Built on Replit.</p></footer>

</body>
</html>
```

Landing Page — CSS

CSS — style.css

```css
/* % %  Reset % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  */
* { margin: 0; padding: 0; box-sizing: border-box; }
body  { font-family: Arial, sans-serif; color: #1e293b; }
a     { text-decoration: none; }

/* % %  Buttons % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  */
.btn-primary {
  background: #3b82f6; color: white; padding: 10px 22px;
  border-radius: 8px; font-weight: bold; border: none; cursor: pointer;
}
.btn-outline {
  border: 2px solid #3b82f6; color: #3b82f6;
  padding: 8px 20px; border-radius: 8px; font-weight: bold;
}
.btn-lg { padding: 16px 36px; font-size: 1.1rem; }

/* % %  Navbar % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  */
.navbar {
  display: flex; align-items: center;
  justify-content: space-between;
  padding: 18px 60px;
  border-bottom: 1px solid #e2e8f0;
  position: sticky; top: 0; background: white; z-index: 100;
}
.nav-logo  { font-size: 1.4rem; font-weight: 900; color: #3b82f6; }
.nav-links { display: flex; gap: 32px; list-style: none; }
.nav-links a { color: #475569; }
.nav-links a:hover { color: #3b82f6; }
.nav-btns  { display: flex; gap: 12px; align-items: center; }

/* % %  Hero % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  */
.hero {
  text-align: center;
  padding: 100px 20px 80px;
  background: linear-gradient(135deg, #eff6ff 0%, #f8fafc 100%);
}
.hero h1 { font-size: 3rem; font-weight: 900; color: #0f172a; }
.hl       { color: #3b82f6; }
.hero p   { font-size: 1.2rem; color: #64748b; margin: 20px 0 36px; }

/* % %  Features % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  */
.features {
  display: flex; gap: 24px; justify-content: center;
  padding: 80px 60px; flex-wrap: wrap;
}
.card {
  background: #f8fafc; border: 1px solid #e2e8f0;
  border-radius: 16px; padding: 36px 28px;
  width: 220px; text-align: center;
}
.icon { font-size: 2.5rem; margin-bottom: 16px; }
.card h3 { font-size: 1.2rem; margin-bottom: 8px; }
.card p  { color: #64748b; font-size: 0.95rem; }

/* % %  Footer % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %  */
footer {
  text-align: center; padding: 40px;
  background: #f1f5f9; color: #94a3b8; font-size: 0.9rem;
```

signup.html — Create Account Page

HTML — signup.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Sign up — MyApp</title>
  <link rel="stylesheet" href="style.css" />
  <link rel="stylesheet" href="auth.css" />
</head>
<body>
  <div class="auth-wrap">
    <div class="auth-card">
      <div class="auth-logo">MyApp</div>
      <h2>Create your account</h2>
      <p class="auth-sub">Join thousands of builders today</p>

      <form id="signupForm" class="auth-form">
        <div class="fg">
          <label for="name">Full name</label>
          <input type="text" id="name" placeholder="Charlie Johnson" required />
        </div>
        <div class="fg">
          <label for="email">Email address</label>
          <input type="email" id="email" placeholder="charlie@example.com" required />
        </div>
        <div class="fg">
          <label for="pwd">Password  (min 8 chars)</label>
          <input type="password" id="pwd" placeholder="••••••••" required />
        </div>
        <button type="submit" class="btn-primary btn-full">
          Create account
        </button>
      </form>

      <p class="auth-foot">
        Already have an account? <a href="login.html">Log in</a>
      </p>
    </div>
  </div>
  <script src="auth.js"></script>
</body>
</html>
```

login.html — Log In Page

HTML — login.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Log in — MyApp</title>
  <link rel="stylesheet" href="style.css" />
  <link rel="stylesheet" href="auth.css" />
</head>
<body>
  <div class="auth-wrap">
    <div class="auth-card">
      <div class="auth-logo">MyApp</div>
      <h2>Welcome back</h2>
      <p class="auth-sub">Log in to your account</p>

      <form id="loginForm" class="auth-form">
        <div class="fg">
          <label for="email">Email address</label>
          <input type="email" id="email" placeholder="charlie@example.com" required />
        </div>
        <div class="fg">
          <label for="pwd">Password</label>
          <input type="password" id="pwd" placeholder="••••••••" required />
        </div>
        <div class="form-row">
          <label><input type="checkbox" /> Remember me</label>
          <a href="#">Forgot password?</a>
        </div>
        <button type="submit" class="btn-primary btn-full">Log in</button>
      </form>

      <p class="auth-foot">No account? <a href="signup.html">Sign up free</a></p>
    </div>
  </div>
  <script src="auth.js"></script>
</body>
</html>
```

auth.css — Styles for both auth pages

CSS — auth.css

```css
.auth-wrap {
  min-height: 100vh;
  background: linear-gradient(135deg, #eff6ff 0%, #f0fdf4 100%);
  display: flex; align-items: center; justify-content: center;
  padding: 40px 20px;
}
.auth-card {
  background: white; border-radius: 20px;
  padding: 48px 44px; width: 100%; max-width: 420px;
  box-shadow: 0 20px 60px rgba(0,0,0,0.08);
  text-align: center;
}
.auth-logo { font-size: 1.6rem; font-weight: 900; color: #3b82f6; margin-bottom: 24px; }
.auth-card h2 { font-size: 1.6rem; font-weight: 800; margin-bottom: 8px; }
.auth-sub  { color: #64748b; margin-bottom: 32px; font-size: 0.95rem; }
.auth-form { display: flex; flex-direction: column; gap: 18px; text-align: left; }
.fg label  { display: block; font-size: 0.875rem; font-weight: 600; color: #374151; margin-bottom: 6px; }
.fg input  {
  width: 100%; padding: 12px 16px;
  border: 1.5px solid #d1d5db; border-radius: 10px;
  font-size: 0.95rem; outline: none;
}
.fg input:focus { border-color: #3b82f6; }
.form-row { display: flex; justify-content: space-between; font-size: 0.875rem; color: #64748b; }
.form-row a { color: #3b82f6; }
.btn-full  { width: 100%; padding: 14px; font-size: 1rem; }
.auth-foot { margin-top: 24px; font-size: 0.9rem; color: #64748b; }
.auth-foot a { color: #3b82f6; font-weight: 600; }
```

auth.js — Form validation and submit logic

JavaScript — auth.js

```
const signupForm = document.getElementById('signupForm');
const loginForm  = document.getElementById('loginForm');

// % %  Signup % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
if (signupForm) {
  signupForm.addEventListener('submit', function (e) {
    e.preventDefault();  // stop page refresh

    const name  = document.getElementById('name').value;
    const email = document.getElementById('email').value;
    const pwd   = document.getElementById('pwd').value;

    if (pwd.length < 8) {
      alert('Password must be at least 8 characters.');
      return;
    }

    // In a real app: send to server. For now, show success.
    alert('Account created! Welcome, ' + name + '!');
    window.location.href = 'login.html';
  });
}

// % %  Login % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
if (loginForm) {
  loginForm.addEventListener('submit', function (e) {
    e.preventDefault();

    const email = document.getElementById('email').value;

    // In a real app: verify against database.
    alert('Logged in as ' + email + '!');
    window.location.href = 'index.html';
  });
}
```

Ø=ÜÅ Your 6 files are ready. Click %¶ Run. In the Preview, you should see your landing page. Click "Log in" and "Sign up free" in the navbar — they should navigate to the other pages.

React — Build a Modern Web App

What is React and when should you use it?

React is a JavaScript library for building interfaces. Instead of one giant HTML file, you split your app into small reusable pieces called "components". When your app has many pages, React keeps things organised. Big companies like Facebook, Airbnb, and Netflix all use React.

'L Plain HTML — gets messy

- Copy navbar on every page
- Fix a bug in 50 places
- Hard to manage 10+ pages

' React — stays organised

- <Navbar /> used everywhere
- Fix once — updates everywhere
- Scales to 100+ pages easily

Create a React project on Replit

1 Click "+ Create Repl" on your dashboard
Go to replit.com, click the create button.

2 Type "React" in the search box
Select the "React" or "React + Vite" template.

3 Name it and click Create
Example: "my-react-app". Wait 5–10 seconds for Replit to set it up.

4 Click %¶ Run
Replit installs all dependencies automatically. After ~30 seconds, a default React page appears in the Preview.

File structure of a React project

Project Structure

```
my-react-app/
% % %  src/
%    % % %  main.jsx          !• Entry point — DO NOT delete or rename
%    % % %  App.jsx           !• Your main component — START HERE
%    % % %  App.css           !• Styles for App
%    % % %  components/       !• Create your own components here
%       % % %  Navbar.jsx
%       % % %  Footer.jsx
% % %  public/
%    % % %  index.html        !• Base HTML — leave it as-is
% % %  package.json          !• Project config
% % %  vite.config.js        !• Build settings — leave as-is
```

App.jsx — Your first complete React component

Open src/App.jsx. Delete everything. Type this:

React JSX — App.jsx

```jsx
import { useState } from 'react';
import './App.css';

function App() {

  // useState creates a variable React watches.
  // When it changes, React automatically re-draws the screen.
  const [count, setCount] = useState(0);
  const [name,  setName]  = useState('');

  return (
    <div className="container">
      <h1>My React App</h1>

      {/* Counter */}
      <div className="card">
        <p>Count: <strong>{count}</strong></p>
        <button onClick={() => setCount(count + 1)}>+ Add 1</button>
        <button onClick={() => setCount(0)}>Reset</button>
      </div>

      {/* Name input */}
      <div className="card">
        <input
          type='text'
          placeholder='Type your name...'
          value={name}
          onChange={(e) => setName(e.target.value)}
        />
        {name && <p>Hello, {name}! Ø=ÜK</p>}
      </div>
    </div>
  );
}

export default App;
```

React — 5 Rules You Must Know

Rule 1 — Component names MUST start with a CAPITAL letter

'L WRONG

```
function navbar() {
  return <nav>...</nav>;
}
```

' CORRECT

```
function Navbar() {
  return <nav>...</nav>;
}
```

Rule 2 — Return ONE parent element (wrap in or <> </>)

'L WRONG — two roots

```
return (
  <h1>Title</h1>
  <p>Text</p>  // CRASH!
);
```

' CORRECT — wrap them

```
return (
  <div>
    <h1>Title</h1>
    <p>Text</p>
  </div>
);
```

Rule 3 — Use {curly braces} to put JavaScript values in JSX

JSX

```jsx
const userName = 'Charlie';
const score    = 42;

return (
  <div>
    <h1>Hello, {userName}!</h1>      {/* shows: Hello, Charlie! */}
    <p>Your score is {score}</p>     {/* shows: Your score is 42 */}
    <p>Double: {score * 2}</p>       {/* shows: Double: 84 */}
  </div>
);
```

Rule 4 — NEVER modify state directly; use the setter function

'L WRONG

```
// Page does NOT update!
count = count + 1;
count++;
```

' CORRECT

```
// Page re-renders instantly
setCount(count + 1);
```

Rule 5 — Events use camelCase: onClick, onChange, onSubmit

```
// 'L WRONG
<button onclick="fn()">Click</button>
```

```
// '  CORRECT
<button onClick={fn}>Click</button>
```

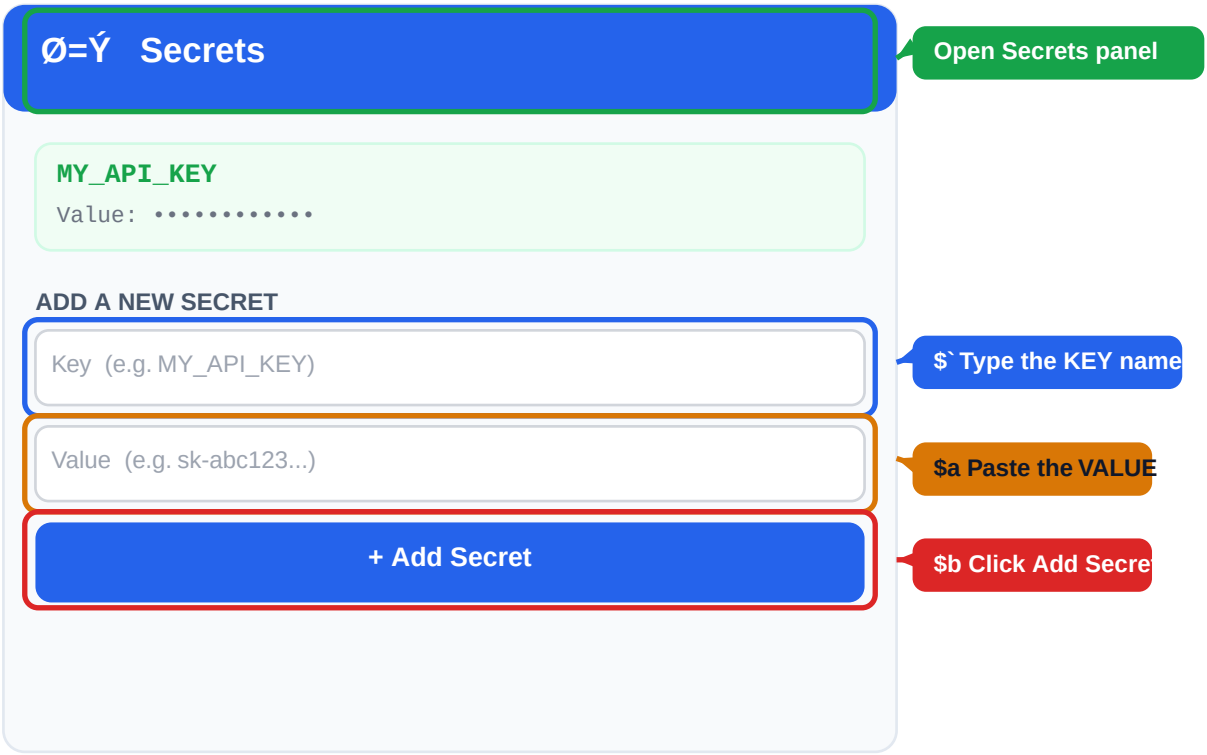
Secrets & Environment Variables

Why this is critical

&Ô NEVER do this: const apiKey = "sk-abc123realkey123"; — if your project is on GitHub or shared, ANYONE can steal it. API keys cost real money.

The right way is to store sensitive values in Replit Secrets. They are encrypted and kept outside your code files.

How to add a Secret — with annotated diagram



How to find the Secrets panel — in the left sidebar

Look at the LEFT ICON BAR in the editor. Find the padlock icon Ø=Ý . It may be labelled "Secrets". Click it to open the panel shown above.

Read your Secret in code

JavaScript / Node.js:

```javascript
// Reads the secret named MY_API_KEY from Replit Secrets
const apiKey = process.env.MY_API_KEY;

if (!apiKey) throw new Error('Missing MY_API_KEY secret!');

fetch('https://api.openai.com/v1/...', {
  headers: { Authorization: `Bearer ${apiKey}` }
})
```

Python:

```python
import os

api_key = os.environ.get('MY_API_KEY')
if not api_key: raise ValueError('Missing MY_API_KEY secret!')

print('API key loaded:', api_key[:6] + '...')  # safe preview
```

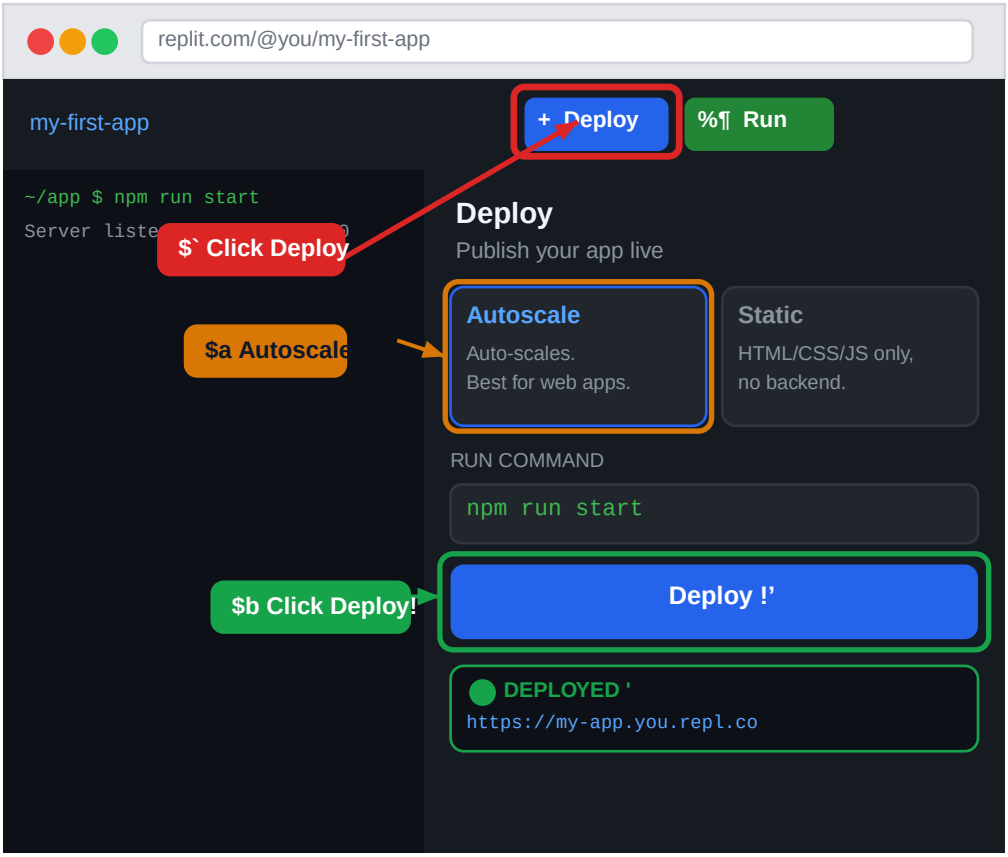
07

Deploy Your App Live

What deploying means

Right now your app runs INSIDE Replit and stops when you close your browser. Deploying publishes it to a permanent server with a real URL. After deploying, the URL works even when your computer is off.

Deploy with Replit — click-by-click with diagram



1 **Click the + Deploy button in the toolbar**
The blue button at the top — highlighted with the red box above.

2 **Select "Autoscale"**
For web apps with a backend (Node.js, Python). Select "Static" for HTML/CSS/JS only sites.

3 **Set the run command**
The command that starts your server. Node.js: npm run start | Python: python main.py

4 **Click the blue Deploy button**
Wait 1–3 minutes. When the green "DEPLOYED" message appears, you are live.

5 **Share your URL**
Copy the repl.co URL and share it. Anyone in the world can now visit your app.

Keyboard Shortcuts Cheat Sheet

Editor shortcuts — use every day

`Ctrl + S`	**SAVE the file**	Use after every few lines. Never forget this.
`Ctrl + Z`	**UNDO last change**	Deleted something by mistake? Press immediately.
`Ctrl + Y`	**REDO (un-undo)**	Undid too much? Redo brings it back.
`Ctrl + A`	**SELECT ALL text**	Select everything in the current file.
`Ctrl + C`	**COPY selection**	Select text first with mouse, then copy.
`Ctrl + X`	**CUT selection**	Like copy but removes the original.
`Ctrl + V`	**PASTE**	Paste at cursor position.
`Ctrl + F`	**FIND in file**	Opens search box — type what you need.
`Ctrl + H`	**FIND + REPLACE**	Change every occurrence of a word at once.
`Ctrl + /`	**COMMENT LINE**	Disables or re-enables the selected line(s).
`Ctrl + D`	**Select next match**	Highlight word !' Ctrl+D selects the next one.
`Ctrl + Enter`	**RUN project**	Same as clicking the %¶ Run button.
`Tab`	**INDENT right**	Moves selected code right (adds spaces).
`Shift + Tab`	**INDENT left**	Moves selected code left (removes spaces).
`Alt + !' / !"`	**MOVE LINE**	Moves current line up or down.
`Ctrl + Shift + P`	**COMMAND PALETTE**	Search for any editor command by name.

Shell commands — must know

`$ node --version`	Check Node.js version
`$ npm install`	Install all packages from package.json
`$ npm install express`	Install a specific package
`$ npm run dev`	Start development server (hot-reload)
`$ npm run build`	Build the app for deployment
`$ npm run start`	Run the production server
`$ ls`	List all files and folders here
`$ cd src`	Enter the folder called "src"
`$ cd ..`	Go back up one folder level
`$ mkdir components`	Create a new folder called "components"

```
$ touch Button.jsx
```

Create a new empty file

```
$ clear
```

Clear the terminal screen

Common Errors & How to Fix Them

How to read an error message

When something breaks, click the Console tab in the right panel. You will see red text. ALWAYS read it carefully:

Ø=Üj In the Console, CLICK on the filename (e.g. "script.js:14") — Replit will jump your cursor to exactly line 14 in the code editor so you can fix it immediately.

The most common errors and how to fix them

ReferenceError: X is not defined

Why: You used a variable before declaring it, or misspelled the name.

```
' WRONG
console.log(myNmae);  // typo — "Nmae" not "Name"
```

```
' FIX
const myName = 'Charlie';
console.log(myName);  // correct spelling
```

SyntaxError: Unexpected token

Why: Missing bracket, parenthesis, or comma. Look at the line number the error shows.

```
' WRONG
function greet( {  // !• missing ) before {
  return "hi";
}
```

```
' FIX
function greet() {
  return "hi";
}
```

TypeError: Cannot read properties of undefined (reading 'name')

Why: You used .name on a variable that is empty (undefined). Check it has a value first.

```
' WRONG
console.log(user.name);  // crashes if user is undefined
```

```
' FIX
if (user && user.name) {
  console.log(user.name);  // safe
}
```

Cannot find module 'express'

Why: You imported a package but forgot to install it. Open the Shell and run npm install.

```
' WRONG
const e = require('express');  // !• crashes, not installed
```

```
' FIX
// In Shell tab, run:
// $ npm install express
// Then your require() will work.
```

Each child in a list needs a "key" prop (React only)

Why: When you use .map() in React, every item needs a unique key attribute.

'L WRONG

```
{items.map(item => <li>{item.name}</li>)}
```

' FIX

```
{items.map(item => <li key={item.id}>{item.name}
</li>)}
```

Quick checklist when something breaks

- Preview is blank !’ click the Console tab and read the red error message.
- Code does not update after editing !’ press Ctrl + S to save, then click %¶ Run again.
- Packages seem missing !’ open the Shell tab and run: npm install
- "command not found: node" !’ your project is Python-based; use python instead.
- Changes not saving !’ check you are editing the right file (look at the tab title).
- Error on "line 14" !’ click that filename in the Console to jump to that line in the editor.

YOU ARE READY TO BUILD

Now go build something! Ø=Þ€

Every great developer started exactly where you are right now.
You have the tools. You have the code. Just start typing.

What you now know how to do:

- Create a Replit account and your first project
- Navigate the editor: files, code, preview, shell
- Write HTML, CSS, and JavaScript from scratch
- Build a landing page with navbar, hero, and features
- Build a signup page and login page with working forms
- Install packages with npm install
- Build a React app with components and useState
- Keep API keys safe using Replit Secrets
- Deploy your app live for anyone to visit
- Read error messages and fix them